Floating-Point Divider Implementation In Verilog Using Newton-Raphson's Approximation

Project Proposal By Group 14

Problem Statement:

Design and simulate a floating-point divider using Newton–Raphson approximation. The method avoids long-division by approximating the reciprocal of the denominator and multiplying it with the numerator. The design should be based on IEEE-754 single-precision floating-point format (32 bits).

Objectives:

---

1. **Implement IEEE-754 single precision floating point representation** handling for 32- bit inputs and outputs.
2. **Design a reciprocal approximation** unit using the Newton-Raphson iterative method.
3. **Implement a floating-point multiplication** and required arithmetic operations.
4. **Generate the final quotient** using: Q = A x (1/B)
5. **Verify the design** through simulation using normal values, negative values, zero, boundary cases and other selected IEEE-754 cases.

Proposed digital logic/design approach:

The proposed divider will operate on two 32-bit IEE-754 single-precision floating- point inputs:

- Dividend(A)
- Divisor(B)

The design will be divided into several stages.

Step 1: IEEE-754 Unpacking

Each 32-bit input will be separated into:

- 1-bit sign
- 8-bit exponent
- 23-bit fraction/mantissa

For normalised numbers, the implicit leading 1 will be included in the mantissa

The IEE-754 format is:

Sign (1 bit)	Exponent (8 bit)	Fraction (23 bit)

Step 2: Divisor Normalization

The divisor will be represented in a form suitable for reciprocal approximation.

Its sign, exponent, and the mantissa will be processed separately so that the Newton-Raphson calculation can operate on the significant portion of the divisor.

Step 3: Initial Reciprocal Approximation

An initial reciprocal approximation (x{0}) for the reciprocal of the normalized divisor will be generated using an 8-case lookup table (LUT). The normalized

divisor mantissa will be divided into eight ranges, and a selected set of its bits will be used as the LUT index.Each of the eight cases will provide a precomputed approximate value of the reciprocal corresponding to that range.

The quality of the initial approximation affects the number of iterations required for convergence.

Step 4: Newton-Raphson iteration

The reciprocal approximation will be refined using:
$X\{n+1\} = X\{n\} (2- BX\{n\})$
Each iteration consists primarily of multiplication and subtraction operations. The number of iterations will be selected based on the desired accuracy and available hardware resource.

Step 5: Multiplication

Once the reciprocal has been calculated, the final division is performed as:

Result= A x (1/B)

The sign, exponent, and mantissa will be processed to obtain the final floating-point result.

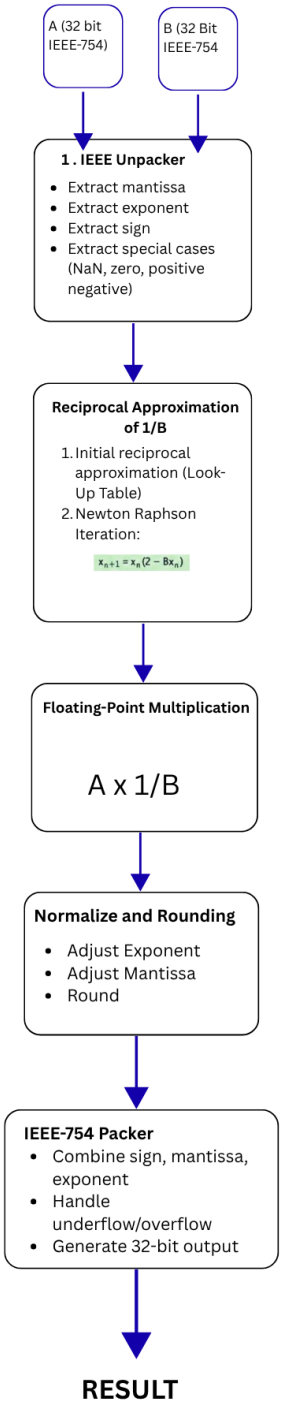
Step 6: Normalization and Rounding

The intermediate result will be normalized and rounded to maintain the required IEEE-754 single-precision representation.

Step 7: IEEE-754 Packing

The resulting sign, exponent and mantissa will be combined to generate the final 32-bit IEEE-754 output.

BLOCK DIAGRAM:



Verification Plan

A Verilog testbench will be made and used to verify the accuracy and functionality of the divider. Some of the categories we will test are as follows:

Category	Example
Exact Division	10/5
Fractional Result	14/3
Result Less Than 1	2/5
Non Exact Division	10/3
Zero Numerator	0/7
Zero Denominator	3/0
Positive/Positive	5/4
Positive/Negative	4/-3
Negative/Positive	-3/7
Negative/Negative	-5/-13
Large Numerator	24605.84/7
Small Numerator	0.125/8
Large Denominator	23/12536.6
Small Denominator	45/0.024

The testbench will check the value obtained from the divider with the expected IEEE-754 result. Also the error between the final hardware output and the expected output will be shown.

Division of work among team members:

Divya Gupta — IEEE-754 Input Processing & Special Cases

- Implement IEEE-754 single-precision input unpacking.
- Extract and process sign, exponent and mantissa.
- Implement detection of zero, infinity, NaN and normal numbers.
- Implement IEEE-754 special-case handling for division.
- Verify input classification and special-case outputs.

Gurnoor Singh — Initial Reciprocal Approximation

- Design the normalized denominator representation.
- Implement the initial reciprocal approximation (x0).
- Design and implement the LUT-based reciprocal estimation.
- Study the effect of LUT size on initial approximation accuracy.
- Verify and analyze the accuracy of different initial estimates.

Monish Chandra P — Newton–Raphson Iteration Unit

- Implement the Newton–Raphson equation:
  x{n+1}=x{n}(2-Bx{n})
- Design the configurable iteration mechanism.
- Implement intermediate floating-point operations required by the iteration.
- Record reciprocal approximation after each iteration.
- Analyze convergence and error reduction with increasing iterations.

Mansi Goyal — Quotient Generation & IEEE-754 Output Processing

- Implement multiplication of numerator by the final reciprocal.
- Perform result normalization and exponent adjustment.

- Implement rounding and final IEEE-754 packing.
- Handle result overflow/underflow where applicable.
- Verify the final 32-bit floating-point quotient for different input combinations.

Keerat Singh Vohra — Control, Integration & Performance Verification

- Design the overall control/FSM and connect all functional modules.
- Manage data flow and configurable iteration control.
- Develop the complete Verilog testbench.
- Compare hardware results with a software/reference model.
- Measure and analyze latency and accuracy for different iteration counts.
- Generate accuracy-vs-iteration and accuracy-vs-latency results.

Shared Final Responsibility

All five members will participate in:

- Full-system integration and debugging.
- Testing with common test cases.
- Final performance analysis.
- Project documentation and presentation.